Where Should Coordination Be Enforced in Multi-Agent Software Engineering?

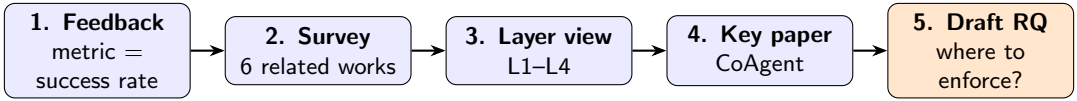
Weekly Research Progress

Chiharu Tomimori

Institute of Science Tokyo

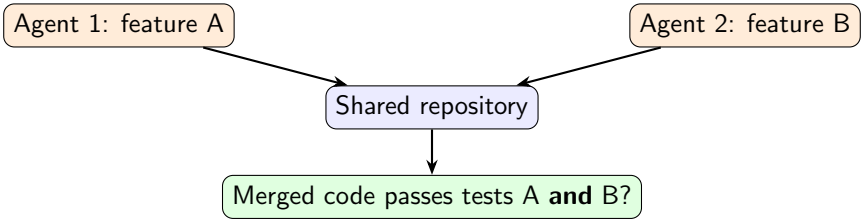
October 23, 2026

This Week at a Glance



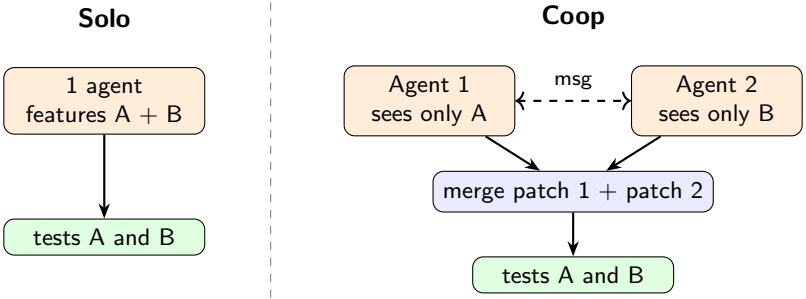
Done this week	Output
Fixed the primary metric	Task success rate (hidden tests pass)
Surveyed related work	CooperBench, CAID, PASC, NP-Bench, CoAgent, position paper
Organized the landscape	Four coordination layers (L1–L4)
Read CoAgent in depth	Strengths, limits, publication status
Narrowed the question	Same protocol, different layers

Problem Setting



- Agents = **concurrent writers** on shared state
- Success = merged result passes **all** tests

CooperBench: Can Two Agents Work as a Team?



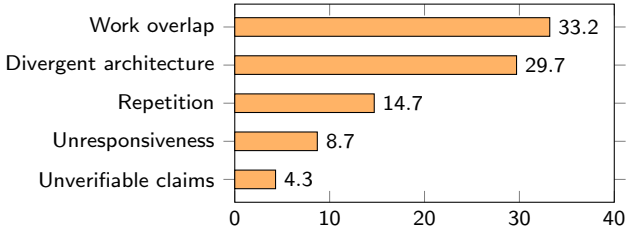
Tasks	652 feature pairs, 12 repos, 4 languages
Built to conflict	features touch the same code
Coordination	prompt + messaging tool only (no enforcement)

A. Khatua, H. Zhu, P. Tran, A. Prabhudesai, F. Sadrieh, J. K. Lieberwirth, X. Yu, Y. Fu, M. J. Ryan, J. Pei, D. Yang. "CooperBench: Why Coding Agents Cannot be Your Teammates Yet." *Conference on Language Modeling (COLM)*, 2026. arXiv:2601.13295.

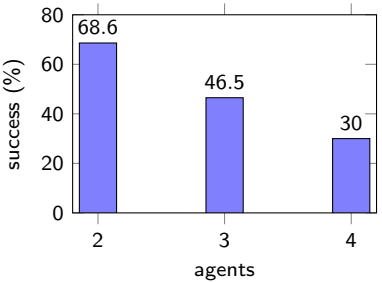
CooperBench: Results

Two agents: ∼25% success (GPT-5, Claude Sonnet 4.5), about **half of solo**. Messaging reduces merge conflicts but not failures.

Failure symptoms (% of failures)

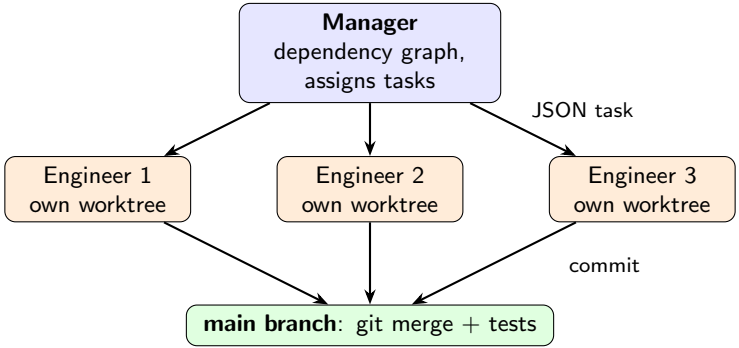


Success vs. number of agents (46 tasks)



A. Khatua, H. Zhu, P. Tran, A. Prabhudesai, F. Sadrieh, J. K. Lieberwirth, X. Yu, Y. Fu, M. J. Ryan, J. Pei, D. Yang. "CooperBench: Why Coding Agents Cannot be Your Teammates Yet." *Conference on Language Modeling (COLM)*, 2026. arXiv:2601.13295.

Closest Work: CAID — Branch and Merge



Parallel edits never touch each other's files directly; conflicts surface at merge time.

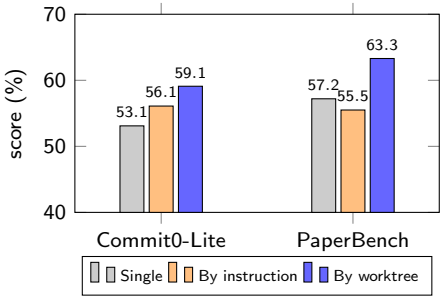
J. Geng, G. Neubig. "Effective Strategies for Asynchronous Software Engineering Agents." arXiv:2603.21489, 2026 (preprint).

CAID: Results

Main result: higher score, higher cost

		Single	CAID
Commit0-Lite	score	53.1	**59.1**
	cost ($)	1.9	8.1
	time (s)	693	1583
PaperBench	score	57.2	**63.3**
	cost ($)	3.3	6.5
	time (s)	1804	2080

Ablation: how to isolate



Claude Sonnet 4.5; 4 engineers (Commit0), 2 (PaperBench)

Limitation: with 8 engineers, two got the same file $\Rightarrow$ 92.1% $\rightarrow$ 44.3% on `simpy`.

J. Geng, G. Neubig. "Effective Strategies for Asynchronous Software Engineering Agents." arXiv:2603.21489, 2026 (preprint).

Existing Work Coordinates at Different Layers

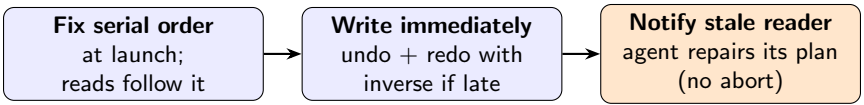
Layer	Mechanism	Example	Writes enforced?
L1 Instruction	"Do not touch others' files" in the prompt	CooperBench coop [1]	×
L2 Advisory tool	Claim / scope tools the agent may ignore	NP-Bench [2]	×
L3 Runtime	Middleware blocks or reorders actions	CoAgent [3]	✓
L4 Workspace / VCS	Separate worktrees, merge with tests	CAID [4]	✓

NP-Bench lists enforced scopes as future work. PASC [5] (shared tree) sees co-edits rise from 16.8% to 53.0% ($2 \rightarrow 5$ agents).

[1] Khatua et al., COLM 2026, arXiv:2601.13295. [2] S. Muku, "Verifying Coordination in Parallel Coding Agents: NP-Bench and a Scheduling Planner," arXiv:2610.07261, 2026. [3] H. Lyu et al., arXiv:2606.15376, 2026. [4] J. Geng, G. Neubig, arXiv:2603.21489, 2026. [5] H. Özaytürk, F. Buzluca, "Coordinating Agents on a Shared Git Workspace: An Empirical Study of Action-Space Observation for Agentic Software Engineering," *ESEM 2026*, LIPIcs vol. 394, 18:1–18:20. doi:10.4230/LIPIcs.ESEM.2026.18.

Key Paper: CoAgent — The Runtime Informs, the Agent Repairs

Problem: agent B builds a canary on an image agent A is fixing ⇒ final state matches **no serial order**



	2PL	OCC
Problem with LLM agents	locks held during inference	abort wastes inference

H. Lyu, D. Zhang, M. Wu, X. Wei, H. Chen. "CoAgent: Concurrency Control for Multi-Agent Systems." arXiv:2606.15376, 2026 (preprint; no peer-reviewed version found as of Oct. 2026).

CoAgent: Results and Limitations

10 contended workloads, 2 agents (relative to serial execution)

Protocol	Speedup	Tokens	Note
No coordination	$\sim$1.5$\times$	–	only 13% of trials correct
2PL	1.04$\times$	–	0.81 deadlocks / trial
OCC	0.93$\times$	1.83$\times$	0.95 aborts / trial
MTPO (proposed)	**1.4**$\times$	**1.15**$\times$	correctness $\sim$5% below serial

Not covered by the evaluation

Coding / git tasks	Prompt-only baseline	>2 agents	>1 model	Free-form bash
$\times$	$\times$	$\times$	$\times$	$\times$

H. Lyu, D. Zhang, M. Wu, X. Wei, H. Chen. "CoAgent: Concurrency Control for Multi-Agent Systems." arXiv:2606.15376, 2026 (preprint).

Gap and Draft Research Question

Study	What is compared	Same protocol across layers?
CoAgent [3]	Protocols inside L3	×
CAID [4]	Isolation: L1 vs. L4	partly
NP-Bench [2]	Planning vs. detection, L2 only	×
Position paper [6]	Argument only, no experiments	×

Research question (draft)

At which layer should a coordination protocol (e.g., scope ownership) be enforced to maximize task success rate?

[2] S. Muku, arXiv:2610.07261, 2026. [3] H. Lyu et al., arXiv:2606.15376, 2026. [4] J. Geng, G. Neubig, arXiv:2603.21489, 2026. [6] X. Yang, L. Li, Z. Ji, T. J. Zhang, W. Jiang. "Position: Multi-Agent Systems Should Prioritize Concurrency Control." *ICML 2026*. arXiv:2608.18092.

Same Protocol, Different Layers: An Example

Protocol (fixed): "Agent 1 owns `a.py`; Agent 2 owns `b.py`. Edit only what you own."

Layer	How the rule is delivered	Agent 1 tries to edit `b.py`	Enforced?
L1 Instruction	Written in the prompt	Edit succeeds	×
L2 Advisory tool	`claim(b.py)` says "owned by Agent 2"	Edit still succeeds	×
L3 Runtime	`b.py` is read-only for Agent 1	`Permission denied`	✓
L4 Workspace	Own worktree; merge rejects out-of-scope files	Edit kept locally, rejected at merge	✓ (late)

Same rule, same owner map, same model and tasks — only **who guarantees the rule** changes.

Planned Experiment: Change Only the Layer

Condition	Layer	Role
Solo (1 agent, both features)	–	reference
Coop	L1	existing
Team mode (advisory claims)	L2	existing
Enforced scopes (file permissions)	**L3**	**proposed**
Worktree + merge (CAID-style)	L4	reference

Fixed	protocol, model, tasks, step budget
Benchmark	CooperBench (gold patches conflict in 76.5% of pairs)
Primary metric	success rate (both features' tests pass)
Secondary	out-of-scope writes, stale-read failures

Risks

Risk	Mitigation
One agent does all the work (serial collapse)	Report parallelism and "rescued by one agent" cases
Stale reads not fixed by write partitioning	Count stale-read failures separately
Blocked agents waste budget	Fix the denial message across runs
Benchmark changed after the paper	Pin one commit of CooperBench
Overlap with AgentRoom, ATM (2026 preprints)	Read them next week

Next Week

1. Run a small CooperBench pilot (10 pairs): solo / coop / team
2. Classify the failures (write conflict, semantic conflict, stale read, capability)
3. Count how often agents write outside their claims in team mode
4. Read AgentRoom and ATM to check overlap

Summary

- Coordinating coding agents lose about **half** of their success
- Existing systems coordinate at **different layers**
- No study we found compares **the same protocol across layers**

Takeaway

The question is not *which protocol*, but *where to enforce it*.

References I

[1] A. Khatua, H. Zhu, P. Tran, A. Prabhudesai, F. Sadrieh, J. K. Lieberwirth, X. Yu, Y. Fu, M. J. Ryan, J. Pei, D. Yang. CooperBench: Why Coding Agents Cannot be Your Teammates Yet. *COLM*, 2026. arXiv:2601.13295.

[2] S. Muku. Verifying Coordination in Parallel Coding Agents: NP-Bench and a Scheduling Planner. arXiv:2610.07261, 2026.

[3] H. Lyu, D. Zhang, M. Wu, X. Wei, H. Chen. CoAgent: Concurrency Control for Multi-Agent Systems. arXiv:2606.15376, 2026.

[4] J. Geng, G. Neubig. Effective Strategies for Asynchronous Software Engineering Agents. arXiv:2603.21489, 2026.

[5] H. Özaytürk, F. Buzluca. Coordinating Agents on a Shared Git Workspace: An Empirical Study of Action-Space Observation for Agentic Software Engineering. *ESEM*, LIPIcs vol. 394, 18:1–18:20, 2026. doi:10.4230/LIPIcs.ESEM.2026.18.

[6] X. Yang, L. Li, Z. Ji, T. J. Zhang, W. Jiang. Position: Multi-Agent Systems Should Prioritize Concurrency Control. *ICML*, 2026. arXiv:2608.18092.